

UNIVERSITEIT•STELLENBOSCH•UNIVERSITY
jou kennisvennoot • your knowledge partner

Design of a universal cooperative collision avoidance system for autonomous vehicles using horizontal manoeuvres only.

by

Adrian Stark
25855670

Mechatronic Project 488

Progress Report

Study leader: Mnr JA Engelbrecht

July 2026

Declaration

I, the undersigned, declare that the work contained in this report is my own original work.

Signature: .....
A. Stark

Date: .....

EXECUTIVE SUMMARY

Student: A. Stark
Supervisor: J.A. Engelbrecht

Title of Project
Design of a universal cooperative collision avoidance system for autonomous vehicles using horizontal manoeuvres only.
Objectives
Model a universal cooperative collision avoidance system and demonstrate it with wheel-based robotic vehicles at scale to show its feasibility.
What is current practice and what are its limitations?
Currently, there are proprietary collision avoidance systems used by a variety of industries and companies, that means that small autonomous applications like consumer drones do not contain this technology. Also, due to their proprietary nature, most collision avoidance systems do not account for other vehicles routes, only addressing the issue as it becomes imminent.
What is new in this project?
This project will aim to create a universal system that can be used across a variety of autonomous vehicles, not designed for one specific case. It will also be cooperative, allowing the vehicles to work together to decide on the most efficient path for all.
If the project is successful, how will it make a difference?
If successful, this project will allow cooperative collision avoidance to be implemented in many more applications, and at any scale, lowering the barrier to entry for the use of this technology.
What are the risks to the project being a success? Why is it expected to be successful?
A model that meets the performance criteria cannot be created. If found, the model may not be demonstrable at small-scale. Time and budgetary risks are also present. All risks are mitigated by planning and making use of all available resources.
What contributions have/will other students made/make?
Previous students have considered cooperative collision avoidance in single applications. Their methods will be considered in the process of this project's completion.
Which aspects of the project will carry on after completion and why?
This project will prove that it is technically feasible, however it must be decided if it is economically feasible to continue and develop the product to deployable scale.
What arrangements have been/will be made to expedite continuation?
The chosen model/method and performance data will be provided. The project will be documented in sufficient detail that someone can scale it up to a deployable state.

Student	Date	Lecturer

Contents

Declaration i

Executive Summary ii

Contents iii

List of Figures v

List of Tables vi

1 Introduction 1
- 1.1 Background . . . 1
- 1.2 Objectives . . . 2

2 Literature Review 3
- 2.1 Background . . . 3
- 2.2 Advantages to Cooperative Collision Avoidance . . . 4
- 2.3 Route Planning . . . 4
  - 2.3.1 A* Search . . . 5
  - 2.3.2 Dijkstra's Algorithm . . . 6
  - 2.3.3 Rapidly-Exploring Random Trees (RRT and RRT*) . . . 6
  - 2.3.4 Artificial Potential Fields . . . 7
  - 2.3.5 D* Lite . . . 8
- 2.4 Evaluation . . . 9
  - 2.4.1 Minimum Safe Distance . . . 9
  - 2.4.2 Path Length Comparison . . . 10
- 2.5 Software Dynamic Modelling . . . 10
  - 2.5.1 DC Motor Model . . . 11
  - 2.5.2 PID Discretisation . . . 11
- 2.6 Hardware for Demonstration . . . 12
  - 2.6.1 Tracking Options . . . 12
  - 2.6.2 Correcting Wide-Angle Lens Distortion . . . 13

3 Model Design Criteria **15**
3.1 Model System Specifications 15
3.2 Model Technical Performances Measures 17

4 Demonstration Design **19**
4.1 Demonstration Specifications 19
4.2 Hardware Design 20
4.3 Interfacing 21

5 Model Selection **23**
5.1 A* Search 23

List of References **27**

List of Figures

4.2.1 Hardware setup 21
4.2.2 Robotic vehicle design 21
4.3.1 Hardware interfacing 22

5.1.1 A* Search with 3 directions 24
5.1.2 A* Search with 5 directions 25
5.1.3 A* Search with 5 directions and no priority 26

List of Tables

3.1.1 Model System Specifications 15
3.2.1 Model Technical Performance Measures 17

4.1.1 Demonstration Specifications 19

Chapter 1

Introduction

1.1 Background

Most vehicles operate in a world in which autonomous and non-autonomously controlled vehicles must inhabit the same space. For example, both traditional cars and newer cars with partial-self driving must use the same roads. This has led to complex collision avoidance systems that only react to the world right in front of them. This means that vehicles can only respond reactively to their surroundings and not proactively. Reacting proactively will make these systems far more effective. If the vehicles involved in each environment can work together to determine the least intrusive paths to their destinations, the need for reactive collision avoidance can be negated. This does require all vehicles involved to be able to communicate with each other but there are scenarios where that is increasingly common, consider using fixed-wing mapping drones all flying over the same area, delivery drones servicing the same area from a distribution hub or carrier robots moving around on a factory floor. Since there are a variety of these scenarios, a universal cooperative collision avoidance system would make it easier for this technology to be effectively implemented in a system. The proposed project is the creation and demonstration of a universal cooperative collision avoidance system for autonomous vehicles. This project, which Mr Adrian Stark is doing as part of a Mechatronic Project 488, stems from a proposal by Prof JAA Engelbrecht. It plans to build on work done by previous master's students (Thando Mtimkulu and Merrick Hughes) who each investigated this technology for a specific use case. This document explains the project's objectives, motivation and planning. The steps that are planned in this process, as well as the expected costs and time scales for the study are also outlined.

1.2 Objectives

As mentioned, the project's aim is to create a cooperative collision avoidance system for autonomous vehicles. The system will make use of the vehicles location, direction, velocity and desired path to determine a new path for each vehicle to its destination with the smallest impact to all vehicles. The system will consider the vehicles minimum velocities and maximum turning rates. It is important that this project will focus on scenarios in which the vehicles involved can slow down but cannot stop rapidly in their path. This is to ensure that the system can be used in applications like fixed wing aircraft or vessels. The project is limited to horizontal manoeuvres so the traditional z-dimension cannot be used as a collision avoidance tactic.
The objectives of this project are therefore:

1. Create and test 3 rudimentary collision avoidance models.
2. Create a simple single-mass simulation of the most feasible model .
3. Build a demonstration of the system using scale robots.

Chapter 2

Literature Review

2.1 Background

Traditional collision avoidance is built on onboard sensing and reactive control. A vehicle senses its surroundings with radar, lidar, ultrasonic, or cameras and converts what it sees directly into a braking or steering command. On the road, this appears as adaptive cruise control and forward-collision warning with automatic emergency braking (Vahidi and Eskandarian, 2003). Most of the difficulty sits in detection, not control: recognising another vehicle has to hold up across changing light, weather, occlusion, and clutter, and every sensor trades range and resolution against cost (Mukhtar *et al.*, 2015).

Planning is usually reactive as well. The Dynamic Window Approach searches a vehicle's feasible velocities over a short horizon and picks whichever keeps it clear of what is nearby while making progress toward the goal (Fox *et al.*, 1997). This runs comfortably in real time, but it optimises over the current sensor snapshot only, with no model of where anything else will be a few seconds later.

Four problems follow from this arrangement. Each vehicle plans from its own point of view with no knowledge of what the others intend, so it has to assume the worst and drive conservatively, sometimes to the point of deadlock. A purely reactive planner can settle into a local minimum or oscillate near another vehicle instead of committing to a manoeuvre. Sensing range and line of sight mean a fast-approaching vehicle outside the field of view stays invisible until it is already close. And when two vehicles each react only to the other, with neither accounting for the fact that the other is also reacting, the result is a back-and-forth that neither one resolves. These four problems are the case for letting vehicles cooperate rather than plan alone.

2.2 Advantages to Cooperative Collision Avoidance

Cooperative collision avoidance (CCA) shares vehicle state and intent so that the work of avoiding a collision is split between vehicles instead of each one guessing at what the others will do.

Velocity-obstacle methods make this concrete. The velocity obstacle of another moving vehicle is the set of a vehicle's own velocities that would eventually put it on a collision course. van den Berg *et al.* (2008) split this evenly between the two vehicles in Reciprocal Velocity Obstacles (RVO), which removes the oscillation that appears when two purely reactive agents both try to dodge in full. van den Berg *et al.* (2011) extend this to Optimal Reciprocal Collision Avoidance (ORCA), writing each pairwise constraint as a half-plane of permitted velocities and intersecting them across all neighbours at once; the result scales to large groups without pairwise oscillation. This is where the case for cooperation is clearest: with many vehicles close together, splitting the avoidance effort produces smoother, collision-free motion that an isolated reactive planner cannot match.

At an intersection, cooperative control uses vehicle-to-vehicle and vehicle-to-infrastructure communication to schedule vehicles through a shared conflict zone rather than leaving each one to brake reactively. Hafner *et al.* (2013) demonstrate this on real test vehicles, coordinating their passage to stay collision-free.

Cooperation is not always the better choice. It helps most in dense traffic, at structured conflicts such as intersections, in a fleet running one shared protocol, and wherever onboard sensing alone falls short. It performs worse when the communication link is the weak point: latency, dropped messages, and limited range all degrade coordination. It also assumes every relevant vehicle is cooperating and behaving predictably, so a single non-cooperating vehicle can void the guarantees, and the extra communication and coordination logic is itself a new source of complexity and failure.

2.3 Route Planning

CCA requires each vehicle to compute a route to its goal that stays clear of the other vehicles' current and predicted positions. Because the only hazards this project plans around are other moving vehicles, no method here needs to encode a fixed map of terrain; instead, every planner has to account for time as well as space, since a given position is only in the way at a particular instant, not permanently. A*, Dijkstra, RRT and RRT*, artificial potential fields, and D* Lite are the methods most often used for this, and each handles the time dimension differently: A* and Dijkstra search a graph or grid and need it extended with a

time axis to represent motion; RRT samples a continuous space and can resample around a moving constraint as it shifts; potential fields react continuously to the other vehicles' current positions and update automatically as they move; and D* Lite repairs an existing plan incrementally as the other vehicles' predicted paths change the cost of a given cell at a given time, rather than solving again from scratch.

2.3.1 A* Search

A* keeps two sets of nodes: an open set of candidates still to be explored, ordered by estimated total cost, and a closed set of nodes already finalised. At each step it removes the open-set node with the lowest

$$f(n) = g(n) + h(n), \tag{2.3.1}$$

where $g(n)$ is the cost of the best path found so far from the start to n and $h(n)$ is a heuristic estimate of the remaining cost from n to the goal, expands its neighbours, and updates g for any neighbour reached more cheaply through n. Provided $h(n)$ never overestimates the true remaining cost — it is *admissible* — the path found the first time the goal is removed from the open set is optimal (Hart *et al.*, 1968). On a grid the heuristic is usually the straight-line distance, $h(n) = \sqrt{(x_n - x_g)^2 + (y_n - y_g)^2}$, or, where only axis-aligned moves are allowed, the Manhattan distance, $h(n) = |x_n - x_g| + |y_n - y_g|$, with (x_g, y_g) the goal cell.

Plain A* searches a fixed map and says nothing about when a cell is occupied. Because the only hazards considered here are other vehicles, each following a predicted trajectory, the search has to run over states (x, y, t) rather than (x, y): a reservation table records which cells are claimed by another vehicle at each future time step, and a neighbour is only expanded if its cell is still free at the time the planning vehicle would reach it (Silver, 2005). This space-time form of A*, often called cooperative A*, is the version that actually applies to a system built to avoid other vehicles rather than fixed terrain.

Advantages: optimal and complete with an admissible heuristic; the heuristic biases the search toward the goal, so far fewer nodes are expanded than an uninformed search would need; straightforward to implement; and weighting the heuristic trades optimality for speed when that is worth doing (Duchoň *et al.*, 2014).

Drawbacks: memory and run time both grow with grid resolution and, once time is added as a third dimension, with the length of the planning horizon; paths follow grid edges and turn at fixed angles, so they usually need smoothing before a wheeled vehicle can track them; and a change to another vehicle's predicted trajectory invalidates the reservation table and forces a fresh search (Duchoň *et al.*, 2014).

2.3.2 Dijkstra's Algorithm

Dijkstra's algorithm is the special case of A* with $h(n) = 0$: every node is expanded purely by its known cost from the start (Dijkstra, 1959). It keeps a priority queue of nodes ordered by $g(n)$ and repeatedly relaxes every edge (u, v) with cost $w(u, v)$ out of the node it removes:

$$\text{if } g(u) + w(u, v) < g(v) \quad \text{then} \quad g(v) \leftarrow g(u) + w(u, v), \tag{2.3.2}$$

re-inserting v with its updated cost. With a binary heap this runs in $O((|V| + |E|) \log |V|)$ for a graph with $|V|$ nodes and $|E|$ edges, and because it carries no heuristic, it explores outward from the start roughly evenly in every direction rather than toward any one goal.

Advantages: it is guaranteed to return the shortest path; it needs no heuristic to be designed or tuned; and because it expands uniformly it produces shortest paths to every node reached, not only one goal, which is useful when several destinations are queried against the same map (Patle *et al.*, 2019).

Drawbacks: with no heuristic to bias it toward a single goal, it typically expands many more nodes than A* for a single start-goal query, which costs time on a large map; and, like A*, a static solve says nothing about when a cell will be occupied, so it inherits the same need to be extended with a time dimension, or re-run whenever the other vehicles' predicted paths change (Patle *et al.*, 2019).

2.3.3 Rapidly-Exploring Random Trees (RRT and RRT*)

RRT builds a tree outward from the start by sampling the configuration space rather than discretising it (LaValle, 1998). Each iteration draws a random sample x_{rand}, finds the tree's nearest existing node x_{near} under a distance metric, and steers from x_{near} toward x_{rand} by at most a fixed step η to produce a candidate node x_{new}; the candidate is added to the tree only if the segment connecting the two is collision-free. Repeating this biases the tree toward unexplored regions of the space, and it terminates once a sample falls within reach of the goal.

RRT* adds a rewiring step so the plan converges toward the optimal path as sampling continues (Karaman and Frazzoli, 2011). Instead of attaching x_{new} to its single nearest neighbour, it considers every existing node within a shrinking radius

$$r = \min \left\{ \gamma \left(\frac{\ln n}{n} \right)^{1/d}, \ \eta \right\}, \tag{2.3.3}$$

where n is the current number of nodes in the tree, d is the dimension of the configuration space, γ is a constant set by the free-space volume, and η is the

maximum step length as before. Among the nodes in this radius, x_{new} is connected to whichever gives it the lowest path cost

$$c(x_{\text{new}}) = c(x_{\text{parent}}) + \|x_{\text{new}} - x_{\text{parent}}\|, \tag{2.3.4}$$

and any neighbour that would itself become cheaper by routing through x_{new} is rewired to do so. Because the only hazards here are other vehicles, the collision check applied to each candidate segment tests it against their predicted positions over the time the vehicle would take to traverse it, rather than against any fixed geometry.

Advantages: both handle high-dimensional and continuous spaces without building a grid; both extend naturally to a vehicle's real turning and acceleration limits by using a steering function that respects those limits instead of straight-line segments; and RRT* converges toward the optimal path as the sample count grows (Karaman and Frazzoli, 2011).

Drawbacks: the raw tree is jagged and usually needs smoothing before it is drivable; plain RRT is feasible but not optimal; RRT*'s rewiring adds a nearest-neighbour and cost-comparison step at every iteration, so convergence to a good path can be slow; and because sampling is random, two runs over the same problem rarely produce the same path (LaValle, 1998).

2.3.4 Artificial Potential Fields

The artificial potential field treats the goal as an attractor and every other vehicle as a repeller, and moves the vehicle down the gradient of their combined potential (Khatib, 1986):

$$U(q) = U_{\text{att}}(q) + U_{\text{rep}}(q), \tag{2.3.5}$$

$$U_{\text{att}}(q) = \tfrac{1}{2}\,\zeta\,\|q - q_{\text{goal}}\|^2, \tag{2.3.6}$$

$$U_{\text{rep}}(q) = \begin{cases} \tfrac{1}{2}\,\eta\left(\dfrac{1}{\rho(q)} - \dfrac{1}{\rho_0}\right)^2, & \rho(q) \le \rho_0, \\ 0, & \rho(q) > \rho_0, \end{cases} \tag{2.3.7}$$

$$F(q) = -\nabla U(q), \tag{2.3.8}$$

where q is the vehicle's own position, q_{goal} its goal, ζ the attractive gain, η the repulsive gain, $\rho(q)$ the distance to the nearest other vehicle, and ρ_0 the range within which that vehicle has any influence at all. The resulting force is converted directly into a steering command at every control step,

$$q_{k+1} = q_k + \alpha\, F(q_k), \tag{2.3.9}$$

with α a step-size gain and k the control-loop index. Because $\rho(q)$ is measured to the other vehicles' current positions, the field updates on its own as they move, with no separate replanning step.

Advantages: cheap to compute and continuous, so it runs comfortably at the control-loop rate rather than needing a discrete search; it reacts to other vehicles as they move with no replanning logic; and the resulting motion is smooth (Khatib, 1986).

Drawbacks: the vehicle can stall in a local minimum where the attractive and repulsive terms cancel, and it can oscillate when squeezed between two vehicles on either side; a goal positioned close to another vehicle's repulsive field can become unreachable; and because the force depends only on the present configuration, it carries no notion of where a vehicle is heading, only where it currently is. Modified potential functions that add an extra term to remove this goal-near-vehicle failure, and rotational terms to help escape local minima, address some of these cases (Ge and Cui, 2000).

2.3.5 D* Lite

D* Lite is an incremental planner for costs that change while a vehicle is already moving toward its goal. Rather than solving the graph again from scratch every time another vehicle's predicted path changes the cost of using a particular cell at a particular time, it repairs only the part of the existing solution that the change actually affects (Stentz, 1994). It keeps two values for every state s: $g(s)$, the current estimate of the cost from s to the goal, and a one-step lookahead value (Koenig and Likhachev, 2002)

$$\mathrm{rhs}(s) = \begin{cases} 0, & s = s_{\mathrm{goal}}, \\ \min\limits_{s' \in \mathrm{Succ}(s)} \big(c(s, s') + g(s')\big), & \text{otherwise}, \end{cases} \tag{2.3.10}$$

where $c(s, s')$ is the cost of moving from s to a successor s', itself raised wherever another vehicle is predicted to occupy s' at the relevant time. A state is *consistent* once $g(s) = \mathrm{rhs}(s)$; any state where the two disagree is queued for re-expansion, ordered by the key

$$k(s) = \Big[\ \min\big(g(s), \mathrm{rhs}(s)\big) + h(s_{\mathrm{start}}, s) + k_m\ ;\ \min\big(g(s), \mathrm{rhs}(s)\big)\ \Big], \tag{2.3.11}$$

compared alphabetically, where $h(s_{\mathrm{start}}, s)$ is the heuristic distance from the vehicle's current state to s and k_m is an offset that accumulates as the vehicle moves so that keys computed earlier stay comparable to new ones. When another vehicle's predicted trajectory changes, only the costs $c(s, s')$ it touches are updated, the

affected states are re-queued, and expansion resumes from the front of the queue rather than from the start node — which is what makes the repair cheap.

Advantages: it repairs a plan quickly as the other vehicles' predicted trajectories are updated, which is the recurring event in this project rather than the exception; and it reuses the previous solve instead of discarding it, which is the main saving over rerunning A* every time a prediction changes (Koenig and Likhachev, 2002).

Drawbacks: it is more involved to implement and keep correct than a plain A* search, since consistency has to be maintained across incremental updates rather than computed once; it is still a grid method and inherits the same path-smoothness problems; and it carries a memory overhead for the stored priority queue and state values between updates (Stentz, 1994).

2.4 Evaluation

Two questions decide whether a route-planning method is fit for CCA: does it keep every pair of vehicles at a safe separation, and how much extra distance does avoidance cost compared with driving straight to the goal. The methods in Section 3 are compared against both criteria over the same set of test trajectories.

2.4.1 Minimum Safe Distance

For vehicles i and j with positions $p_i(t), p_j(t) \in R^2$, the separation at time t is

$$d_{ij}(t) = \|p_i(t) - p_j(t)\|. \tag{2.4.1}$$

A trial is collision-free only if $d_{ij}(t) \geq d_{\text{safe}}$ for every pair and every t during the run. The safe distance is not a single fixed number; it is built from the vehicles' physical size and their ability to stop or turn away:

$$d_{\text{safe}} = r_i + r_j + v_{\text{rel}}\, t_r + \frac{v_{\text{rel}}^2}{2\, a_{\max}}, \tag{2.4.2}$$

where r_i and r_j are the bounding radii of the two vehicles, $v_{\text{rel}} = \|v_i(t) - v_j(t)\|$ is their relative speed, t_r is the combined sensing-and-actuation reaction time, and $a_{\max}$ is the maximum deceleration either vehicle can apply. The last two terms are the same reaction-distance-plus-braking-distance construction used in adaptive cruise control headway design (Vahidi and Eskandarian, 2003), applied here between two moving vehicles rather than between a vehicle and the one ahead of it. The minimum separation actually achieved during a run,

$$d_{\min} = \min_{t} \min_{i \neq j} d_{ij}(t), \tag{2.4.3}$$

is recorded for every trial and every planner; $d_{\min} \geq d_{\text{safe}}$ is a pass, and anything below it is a near-miss or collision. Because both the RVO and ORCA formulations already define a collision in terms of the combined radii of the two vehicles (van den Berg *et al.*, 2008, 2011), this criterion is directly compatible with cooperative planners evaluated the same way.

2.4.2 Path Length Comparison

Safety alone does not capture how much a planner disturbs a vehicle's route. Because the only hazards in this project are other vehicles, the shortest possible path for a vehicle acting alone is simply the straight line to its goal,

$$L_0 = \|p_{\text{goal}} - p_{\text{start}}\|. \tag{2.4.4}$$

The length of the path a vehicle actually drives once it has to avoid the other vehicles is the arc length of its logged trajectory,

$$L = \int_{t_0}^{t_f} \|\dot{p}(t)\|\, dt = \int_{t_0}^{t_f} \sqrt{\dot{x}(t)^2 + \dot{y}(t)^2}\, dt, \tag{2.4.5}$$

where $p(t) = (x(t), y(t))$ is the vehicle's position and t_0, t_f are the start and end times of the run. Simulink and the tracking hardware in Section 6 both produce sampled data rather than a continuous function, so L is evaluated as a discrete sum over the N logged samples,

$$L \approx \sum_{k=0}^{N-1} \|p_{k+1} - p_k\|. \tag{2.4.6}$$

The difference between the two,

$$\Delta L = L - L_0, \tag{2.4.7}$$

is the extra distance the avoidance manoeuvre cost, and the ratio $\eta = L_0/L$ gives a planner-independent efficiency score between 0 and 1, with $\eta = 1$ meaning the vehicle reached its goal without deviating at all. Running the same start/goal pairs and the same set of other-vehicle trajectories through each planner in Section 3 and comparing their $d_{\min}$ and η values side by side — together with the planning and replanning times already discussed for each method — is what decides which one is actually used.

2.5 Software Dynamic Modelling

The wheeled vehicles are driven by DC motors, with each wheel's speed regulated by a digital PID controller. Both are modelled in Simulink, working up from the motor to the discrete controller that drives it.

2.5.1 DC Motor Model

A brushed DC motor is described by one electrical and one mechanical equation (Kluever, 2019). The armature circuit gives

$$V_a(t) = R_a\, i_a(t) + L_a \frac{di_a(t)}{dt} + K_e\, \omega(t), \tag{2.5.1}$$

where V_a is the applied armature voltage, R_a and L_a are the armature resistance and inductance, i_a is the armature current, ω is the rotor angular velocity, K_e is the back-EMF constant, and $K_e\,\omega$ is the back-EMF opposing the applied voltage. This current produces a torque

$$T_m(t) = K_t\, i_a(t), \tag{2.5.2}$$

with K_t the torque constant, and the mechanical side — rotor plus load — follows

$$J\frac{d\omega(t)}{dt} + b\,\omega(t) = T_m(t) - T_L(t), \tag{2.5.3}$$

where J is the combined inertia, b the viscous friction coefficient, and T_L the load torque. In consistent SI units the torque and back-EMF constants are equal, $K_t = K_e = K$. Taking Laplace transforms with $T_L = 0$ and eliminating i_a gives the voltage-to-speed transfer function used directly in Simulink,

$$\frac{\omega(s)}{V_a(s)} = \frac{K}{(L_a s + R_a)(Js + b) + K^2}, \tag{2.5.4}$$

(Kluever, 2019; Franklin *et al.*, 2019). Where L_a is small relative to the mechanical time constant it is often dropped, reducing the model to first order.

2.5.2 PID Discretisation

The continuous PID controller is

$$u(t) = K_p\, e(t) + K_i \int_0^t e(\tau)\, d\tau + K_d \frac{de(t)}{dt}, \qquad C(s) = K_p + \frac{K_i}{s} + K_d s, \tag{2.5.5}$$

where $e(t) = r(t) - y(t)$ is the error between the reference r and the measured output y, and K_p, K_i, K_d are the proportional, integral, and derivative gains (Franklin *et al.*, 2019). A digital implementation runs at a fixed sample period T, replacing the operator s with a discrete approximation. The common substitutions are

$$\text{Forward Euler:} \quad s \leftarrow \frac{z-1}{T}, \tag{2.5.6}$$

$$\text{Backward Euler:} \quad s \leftarrow \frac{z-1}{Tz}, \tag{2.5.7}$$

$$\text{Tustin (bilinear):} \quad s \leftarrow \frac{2}{T}\,\frac{z-1}{z+1}. \tag{2.5.8}$$

Applying backward differences to the integral and derivative terms gives the positional difference equation

$$u[k] = K_p\, e[k] + K_i T \sum_{j=0}^{k} e[j] + \frac{K_d}{T}\big(e[k] - e[k-1]\big), \tag{2.5.9}$$

and differencing this once more gives the incremental form actually run on the controller,

$$\Delta u[k] = K_p\big(e[k] - e[k-1]\big) + K_i T\, e[k] + \frac{K_d}{T}\big(e[k] - 2e[k-1] + e[k-2]\big), \tag{2.5.10}$$

where k is the sample index. The incremental form avoids storing the running error sum and handles integrator windup and controller switching more cleanly, which is why it is the common choice in embedded code (Franklin *et al.*, 2019). Tustin's substitution is usually preferred for the integral and derivative terms because it maps the entire left-half s-plane inside the unit circle, so a continuous design that is stable stays stable once discretised (Franklin *et al.*, 2019). Choosing the sample period T relative to the motor's own time constants is what keeps the discrete controller a faithful match to the continuous design it was tuned from (Kluever, 2019).

2.6 Hardware for Demonstration

Testing the collision avoidance system needs each vehicle's pose — position and heading — in a shared reference frame, in real time. The options span dedicated motion capture, camera-based fiducial markers, radio ranging, and onboard odometry.

2.6.1 Tracking Options

Motion capture (Vicon, OptiTrack). Several infrared cameras track reflective markers on each vehicle and reconstruct its pose to sub-millimetre accuracy at a high update rate (Merriaux *et al.*, 2017), which is why such systems are the usual choice for ground truth. The cost is in the cameras themselves, a fixed and carefully calibrated capture volume, and reflective markers fitted to every vehicle. This is expensive and not portable, and more infrastructure than a small demonstration generally needs.

ArUco fiducial markers. A single overhead camera detects printed square markers, and each marker's identifier and four corners together give location and

orientation. ArUco's marker dictionaries are chosen so that valid patterns sit far apart in bit-pattern space, giving reliable identification with built-in error correction, and detection holds up under partial occlusion (Garrido-Jurado *et al.*, 2014); a later revision of the detector cuts detection time enough for real-time use (Romero-Ramirez *et al.*, 2018). The method is cheap, printed paper and one camera, and because every marker carries an identifier , several vehicles are told apart automatically. Its accuracy depends on camera resolution, marker size, and lighting; the pose is noisier than motion capture; corner detection degrades under motion blur and at shallow viewing angles; and it needs an unbroken line of sight. In practice, ArUco offers an open-source module inside OpenCV (`cv2.aruco`) with maintained ROS drivers, so marker generation, camera calibration, and pose estimation reduce to a handful of function calls, the main reason to choose it for a demonstration system.

AprilTag fiducial markers. AprilTag works on the same principle with a different tag family. Its detector has a very low false-positive rate (Olson, 2011), and a second version improves both detection speed and range (Wang and Olson, 2016). Detection is robust, accurate, and open source. Against that, the original version was heavier to compute than ArUco (the second version narrows that gap), a tag encodes fewer identifier bits for a given physical size, and it shares ArUco's line-of-sight and lighting constraints.

Ultra-wideband ranging. Fixed anchors and a tag on each vehicle measure signal time-of-flight and trilaterate position (Alarifi *et al.*, 2016). This needs no camera line of sight, is unaffected by lighting, and covers a larger area than a single overhead camera can. Its drawbacks: a single tag gives position but not heading, so a second tag or sensor fusion is needed; accuracy sits at the decimetre level; indoor multipath introduces error; and every anchor and tag adds hardware cost and its own calibration.

Wheel encoders. Integrating wheel-encoder counts gives a pose estimate with no external infrastructure at all, at a high rate and low cost. Because it integrates, wheel slip and small measurement errors accumulate into unbounded drift, and it carries no absolute reference. It is therefore used to smooth and fill in between updates from one of the methods above, not as the sole source of pose.

2.6.2 Correcting Wide-Angle Lens Distortion

An overhead camera wide enough to cover the whole test area typically uses a wide-angle, short-focal-length lens, which introduces strong radial (barrel) distortion:

straight lines bow outward and marker corners shift, most severely near the edges of the frame, biasing any pose estimate taken there. The fix is camera calibration: estimating the lens parameters once and undistorting every frame before marker detection runs. The Brown–Conrady model relates an ideal, normalised image point (x, y) to the distorted point (x_d, y_d) the sensor actually records:

$$x_d = x\,(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + 2p_1 xy + p_2(r^2 + 2x^2), \tag{2.6.1}$$

$$y_d = y\,(1 + k_1 r^2 + k_2 r^4 + k_3 r^6) + p_1(r^2 + 2y^2) + 2p_2 xy, \tag{2.6.2}$$

with $r^2 = x^2 + y^2$. The coefficients k_1, k_2, k_3 describe radial distortion and p_1, p_2 describe tangential distortion from lens misalignment. Zhang's method fits these coefficients, together with the camera's intrinsic matrix, from several views of a planar checkerboard (Zhang, 2000), and OpenCV implements both the calibration routine and the inverse mapping that undistorts each frame (Bradski, 2000). Calibrating once and undistorting every frame afterward removes most of the edge-of-frame error and keeps the ArUco pose estimate consistent across the whole field of view. For an extreme fisheye lens a dedicated fisheye model replaces the polynomial above.

Chapter 3

Model Design Criteria

3.1 Model System Specifications

To design the route planning software and to be able to verify it's functionality with a single-mass model in Simulink, the system requirements need to be determined. Some are a function of the limits placed upon this topic; these include the use of only horizontal manoeuvres, and operating above minimum speed. Others define how the system will be tuned; what limits will be placed upon the system to achieve the desired outcomes.

Table 3.1.1: Model System Specifications

Spec ID	**Description**	**Requirement**
SS_1	Allowable control inputs to vehicles.	Horizontal only.
SS_2	Immovable objects in movement area.	None.
SS_3	Movement area.	Square open area for vehicles to move along paths. Vehicles will enter through a boundary and leave through a boundary. It is a small snap-shot of a longer path.
SS_4	Path restrictions within movement area.	Only limited by 2D vehicle dynamics and operation $\geq$ minimum speed.

continued on next page

Table 3.1.1 – continued from previous page

Spec ID	Description	Requirement
SS_5	Movement area size.	A square. 10x the length of the largest vehicles length. Will use $x - y$ coordinates.
SS_6	Collision avoidance system inputs.	Vehicle location, sense, current velocity, minimum velocity and maximum turn rate.
SS_7	Vehicle entry and exit points.	Vehicles can enter and exit on any boundary and on the same boundary.
SS_8	Control system tuning.	System will be tuned to achieve the largest distance between vehicles and lowest control input/smallest deviation from path. Smallest deviation from non-interrupted path will also be considered.
SS_9	System vehicle restrictions.	None. As long as all control system input parameters are known.
SS_10	Trajectory types.	Trajectories will be discretized paths that will abide by the speed and turn rate restrictions of the vehicles. It is assumed that the calculation time is negligible.
SS_11	Trajectory control.	The system will compute the path for all vehicles that contain no collisions. The paths will be outputted for the demonstration vehicles to follow.
SS_12	Trajectory setting.	Randomized start and trajectory setting (within vehicle dynamic restrictions). Edge cases will be preprogrammed.
SS_12	Control system evaluation	Success rate $\geq 99\%$. Path deviation and energy input required. Computational time < 1s.

continued on next page

Table 3.1.1 – continued from previous page

Spec ID	Description	Requirement
SS_13	Safety Buffer/Minimum vehicle interaction distance.	Shortest distance = 1x width of vehicle.
SS_14	Well-clear collisions.	Collisions were the minimum vehicle interaction is observed.
SS_15	Maximum number of vehicles	Practical limitation of a project of this scope.

3.2 Model Technical Performances Measures

The built model must be evaluated to see if it is viable and can be safely implemented. To do this, a set of technical performance measures (TPM) have been determined.

Table 3.2.1: Model Technical Performance Measures

TPM ID	Description	Target range	Motivation
M_TPM_01	Success rate (edge cases)	100%	The success of a collision avoidance system is that a collision never happens.
M_TPM_02	Success rate (randomized)	100%	The success of a collision avoidance system is that a collision never happens.
M_TPM_03	Path updates per iteration.	≤ 1	The model needs to get it right first time and not update it again. There will be no position feedback in the model.
M_TPM_04	Minimum safe distance.	1x Vehicle width	Ensures comfort for passenger vehicles.

continued on next page

Table 3.2.1 – continued from previous page

TPM ID	**Description**	**Target range**	**Motivation**
M_TPM_05	Maximum path deviation	1x Vehicle width	Same as minimum safe distance, as if exceeded, the paths would no longer avoid collisions.
M_TPM_06	Vehicle path integral	Lower is better	The deviation between the normal path a vehicle would take and the one taken
M_TPM_07	Maximum allowable acceleration (well-clear).	$\leq$ 1.5g	To ensure the quality of experience for a passenger vehicle.
M_TPM_08	Maximum allowable jerk (well-clear).	$\leq$ 5.0 m/s^2	To ensure the quality of experience for a passenger vehicle.
M_TPM_09	Well-clear (edge cases)	85 - 100%	Certain scenarios mean that well-clear cannot be achieved, but it is desirable to occur as infrequently as possible.
M_TPM_10	Well-clear (randomized)	95 - 100%	Certain scenarios mean that well-clear cannot be achieved, but it is desirable to occur as infrequently as possible.

Chapter 4

Demonstration Design

4.1 Demonstration Specifications

Due to the scope of this final-year project, it is important to define what the demonstration will do such that the project is achievable within the time.
The cooperation between vehicles for this demonstration will not be active. The paths will be determined centrally on a laptop, where the MATLAB code can be run to generate them. From there, the paths will be sent over Bluetooth to the robots.

Table 4.1.1: Demonstration Specifications

Spec ID	**Description**	**Requirements**
DS_1	Vehicle cooperation.	Passive. Vehicles do not have to actively broadcast location, sense, current velocity, minimum velocity and maximum turn rate. Will be measured externally.
DS_2	Movement area.	A square marked on the floor by Aruco markers. With a 145° FOV camera, it will be 13.60x10.39m.

continued on next page

Table 4.1.1 – continued from previous page

Spec ID	Description	Requirements
DS_3	Collision avoidance system inputs.	Aruco tracking will be used to determine location, sense and velocity. Will be used as a feedback control system to correct if the robots are not in the correct position.
DS_4	Computing location.	Non-distributed and non-collaborative computing. Will be done on a student laptop.
DS_5	Trajectory types.	Paths will be discretized only updating states at 5Hz.
DS_6	Trajectory control.	Trajectories will be preprogrammed before the test starts. Once started, the system will compute the path for all vehicles that contain no collisions. The vehicles will then execute this path. This path commands given to the vehicles will be changed if they are off course.

4.2 Hardware Design

A ceiling mounted wide field of view (FOV) camera is used to track the vehicles in the movement area using the attached ArUco markers seen in Figure 4.2.1. The movement area is marked by ArUco markers on each corner, which provide reference positions and will also be used to correct the image distortion of the camera. The vehicles' location data, tracked over time will be used to determine velocities and accelerations.

Each vehicle uses an Arduino R3 microcontroller, an HC-06 Bluetooth receiver for command input, and is powered by a three cell Li-Po battery. The vehicles employ a tank-drive system that varies forward wheel speeds to achieve different turn radii and optical encoders provide wheel speed measurements. This can be seen in4.2.2.

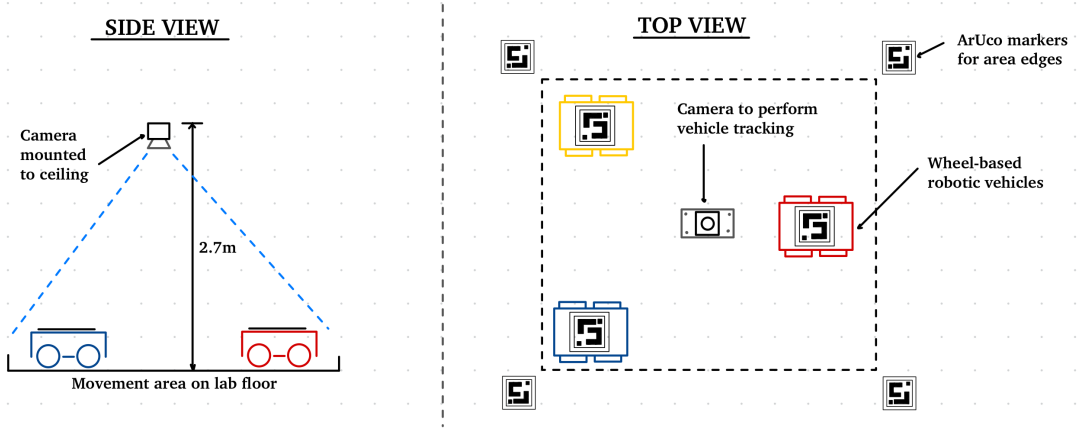


Figure 4.2.1: Hardware setup

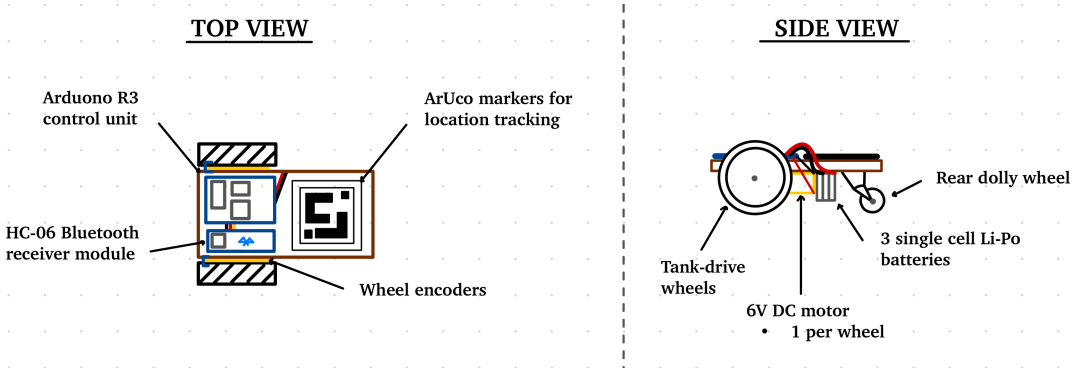


Figure 4.2.2: Robotic vehicle design

4.3 Interfacing

The robots are placed in the movement area, but their placed position will not be identical for each run, so their position is measured to ensure the calculated path starts at their actual location. The collision-free paths that the vehicles must follow are then calculated. The robot control script converts the path .csv file into commands that are sent by Bluetooth to the respective robotic vehicles to instruct them how to move. The HC-06 Bluetooth module allows the path commands to be sent from the PC to the Arduino. The Arduinos use the commands to regulate the PWM signal duty cycle, with PID control from the optical encoders, to maintain the trajectory. The camera feed is converted into $(x; y)$ co-ordinates for each marker, using an open-source driver. If a vehicle is deviating from its path, the control script will adjust the path command to that vehicle so that it maintains its heading.

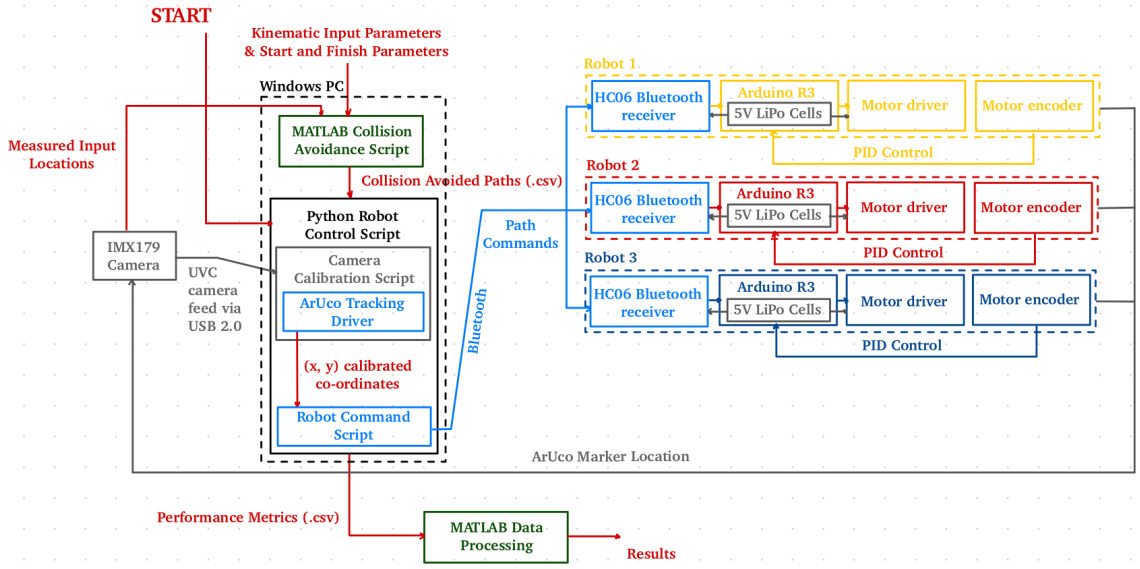


Figure 4.3.1: Hardware interfacing

Chapter 5

Model Selection

5.1 A* Search

For the first and most rudimentary model used, a basic A* Search was used for it's low computational time and ease of implementation. Though the computational time scales with the increase in grid resolution, it was a tradeoff worth considering. As a basis, it made use of a 3-direction system, so the vehicles could only move left, forward or right. This makes for a segmented path from the start-node to the finish-node, with 90° corners that a real-life vehicle could not follow. Furthermore, this is limited to perpendicular entries and exits out of the movement area.
Thus, to make it followable by a vehicle, the path is smoothed by applying a inner minimum turn radius to the corner to make a followable path. This turn is done at the lowest speed of the vehicle.
It provides a viable, yet poorly optimised solution to the problem. On this simplified test scenario, the success of the the model can be seen in Figure 5.1.1.

Following on from that, to improve the path commands, the number of directions was increased to 5, resulting in much shorter paths seen in Figure 5.1.2.
Furthermore, to make the model functional useful and cooperative, it was brute-forced through repetitions to select the scenario in which the smallest increase in travel distance for each vehicle was achieved. For the displayd start nodes, this can be seen in Figure 5.1.3.
This implementation shows that with the adoption of safe guards such as minimum well-clear distances and maximum accelerations and jerks, this could be a viable model though doesn't produce the shortest paths and if high resolution is required, may require non-negligible computation time.

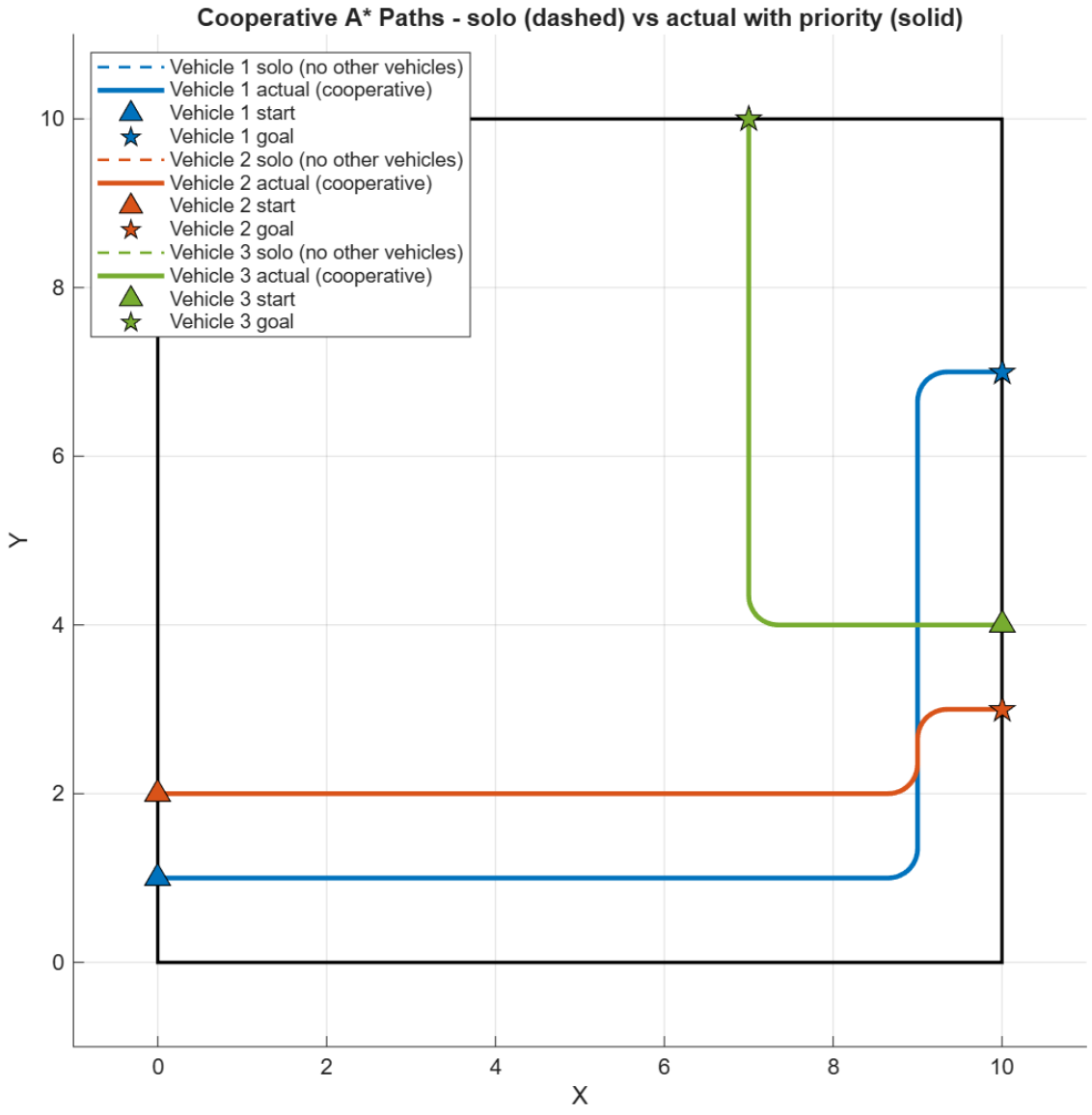


Figure 5.1.1: A* Search with 3 directions

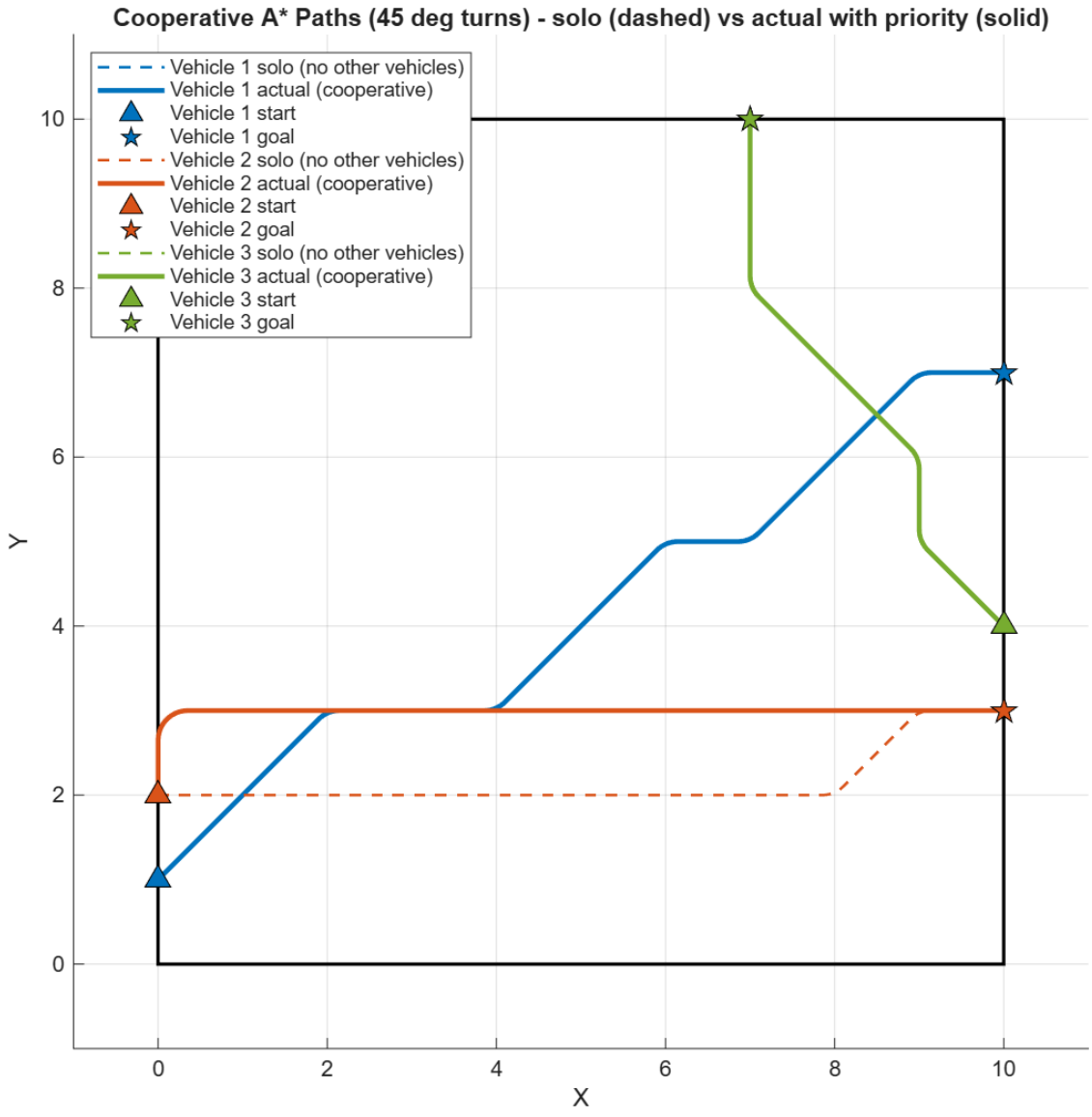


Figure 5.1.2: A* Search with 5 directions

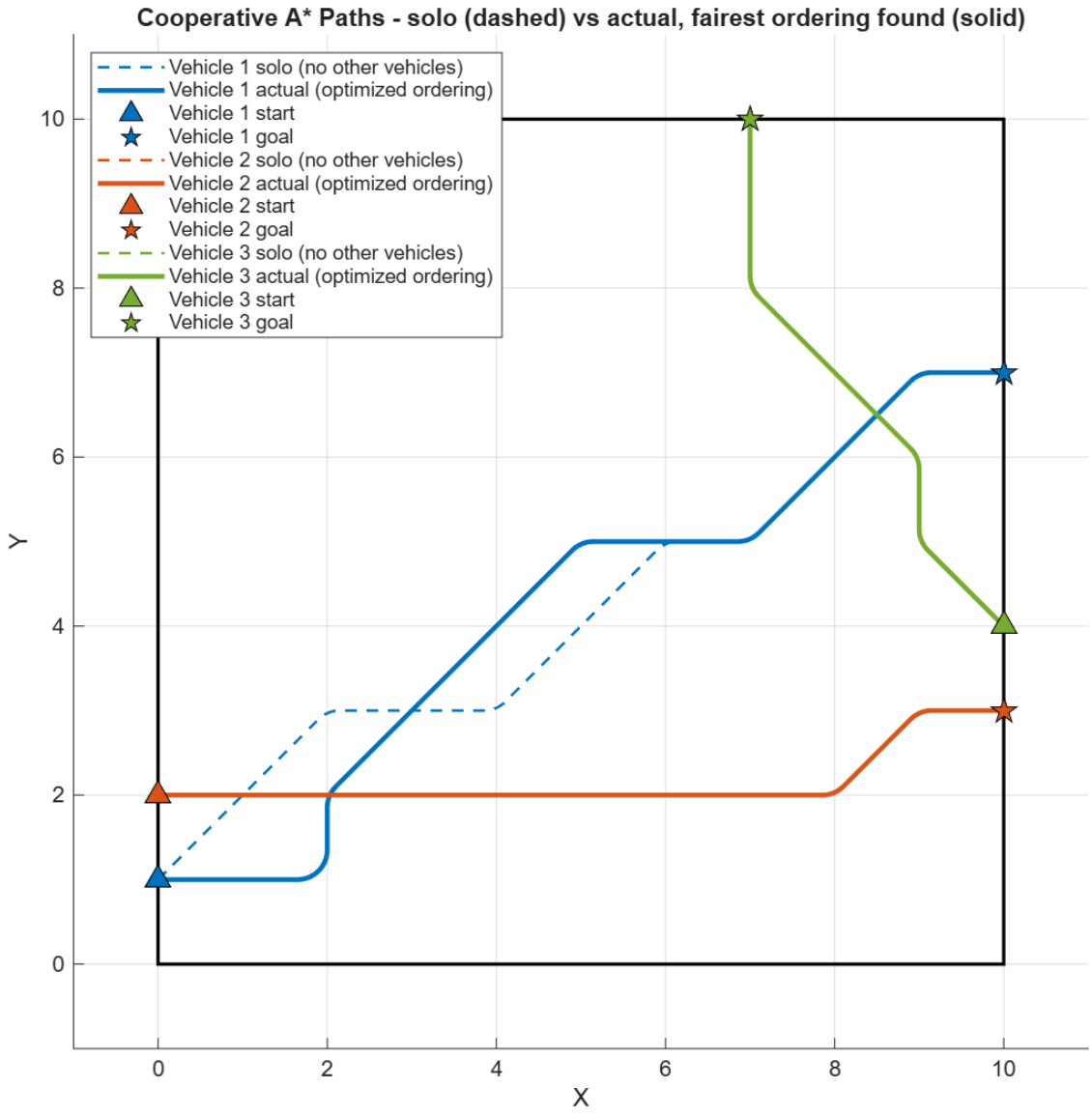


Figure 5.1.3: A* Search with 5 directions and no priority

List of References

Alarifi, A., Al-Salman, A., Alsaleh, M., Alnafessah, A., Al-Hadhrami, S., Al-Ammar, M.A. and Al-Khalifa, H.S. (2016). Ultra wideband indoor positioning technologies: Analysis and recent advances. *Sensors*, vol. 16, no. 5, p. 707.

Bradski, G. (2000). The OpenCV library. *Dr. Dobb's Journal of Software Tools*.

Dijkstra, E.W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, vol. 1, no. 1, pp. 269–271.

Duchoň, F., Babinec, A., Kajan, M., Beňo, P., Florek, M., Fico, T. and Jurišica, L. (2014). Path planning with modified a star algorithm for a mobile robot. *Procedia Engineering*, vol. 96, pp. 59–69.

Fox, D., Burgard, W. and Thrun, S. (1997). The dynamic window approach to collision avoidance. *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, pp. 23–33.

Franklin, G.F., Powell, J.D. and Emami-Naeini, A. (2019). *Feedback Control of Dynamic Systems*. 8th edn. Pearson.

Garrido-Jurado, S., Muñoz-Salinas, R., Madrid-Cuevas, F.J. and Marín-Jiménez, M.J. (2014). Automatic generation and detection of highly reliable fiducial markers under occlusion. *Pattern Recognition*, vol. 47, no. 6, pp. 2280–2292.

Ge, S.S. and Cui, Y.J. (2000). New potential functions for mobile robot path planning. *IEEE Transactions on Robotics and Automation*, vol. 16, no. 5, pp. 615–620.

Hafner, M.R., Cunningham, D., Caminiti, L. and Del Vecchio, D. (2013). Cooperative collision avoidance at intersections: Algorithms and experiments. *IEEE Transactions on Intelligent Transportation Systems*, vol. 14, no. 3, pp. 1162–1175.

Hart, P.E., Nilsson, N.J. and Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107.

Karaman, S. and Frazzoli, E. (2011). Sampling-based algorithms for optimal motion planning. *The International Journal of Robotics Research*, vol. 30, no. 7, pp. 846–894.

Khatib, O. (1986). Real-time obstacle avoidance for manipulators and mobile robots. *The International Journal of Robotics Research*, vol. 5, no. 1, pp. 90–98.

Kluever, C.A. (2019). *Dynamic Systems: Modeling, Simulation, and Control*. 2nd edn. John Wiley & Sons.

Koenig, S. and Likhachev, M. (2002). D* lite. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 476–483.

LaValle, S.M. (1998). Rapidly-exploring random trees: A new tool for path planning. Tech. Rep. TR 98-11, Computer Science Department, Iowa State University.

Merriaux, P., Dupuis, Y., Boutteau, R., Vasseur, P. and Savatier, X. (2017). A study of vicon system positioning performance. *Sensors*, vol. 17, no. 7, p. 1591.

Mukhtar, A., Xia, L. and Tang, T.B. (2015). Vehicle detection techniques for collision avoidance systems: A review. *IEEE Transactions on Intelligent Transportation Systems*, vol. 16, no. 5, pp. 2318–2338.

Olson, E. (2011). Apriltag: A robust and flexible visual fiducial system. In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3400–3407.

Patle, B.K., Ganesh Babu, L., Pandey, A., Parhi, D.R.K. and Jagadeesh, A. (2019). A review: On path planning strategies for navigation of mobile robot. *Defence Technology*, vol. 15, no. 4, pp. 582–606.

Romero-Ramirez, F.J., Muñoz-Salinas, R. and Medina-Carnicer, R. (2018). Speeded up detection of squared fiducial markers. *Image and Vision Computing*, vol. 76, pp. 38–47.

Silver, D. (2005). Cooperative pathfinding. In: *Proceedings of the First AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE)*, pp. 117–122.

Stentz, A. (1994). Optimal and efficient path planning for partially-known environments. In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3310–3317.

Vahidi, A. and Eskandarian, A. (2003). Research advances in intelligent collision avoidance and adaptive cruise control. *IEEE Transactions on Intelligent Transportation Systems*, vol. 4, no. 3, pp. 143–153.

van den Berg, J., Guy, S.J., Lin, M. and Manocha, D. (2011). Reciprocal n-body collision avoidance. In: *Robotics Research: The 14th International Symposium ISRR*, vol. 70 of *Springer Tracts in Advanced Robotics*, pp. 3–19. Springer.

van den Berg, J., Lin, M. and Manocha, D. (2008). Reciprocal velocity obstacles for real-time multi-agent navigation. In: *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pp. 1928–1935.

Wang, J. and Olson, E. (2016). Apriltag 2: Efficient and robust fiducial detection. In: *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 4193–4198.

Zhang, Z. (2000). A flexible new technique for camera calibration. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 22, no. 11, pp. 1330–1334.